PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

Actividad Express y Middleware

Presentado a: Yoly Alexandra Guevara Muñoz

Por Aprendiz: Miguel Angel Rivera

Ficha: 3312932

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día 01 de mes 10 del año 2026

4. investigación:

1. Un middleware es una función intermedia que se ejecuta en el recorrido que hace una petición HTTP desde que llega al servidor hasta que este devuelve una respuesta. Su función es atrapar la solicitud para procesarla, validarla o

modificarla. Se ejecuta de manera secuencial en el ciclo de vida de la petición, en el orden en que fue declarada en el código.

2.
   - **req:** es un request o petición,el cual es un objeto que contiene toda la información enviada por el cliente hacia el servidor, como los datos del body, los parámetros de la URL, los encabezados y el método HTTP.
   - **res:** es un response o respuesta,el cual es el objeto que utiliza el servidor para construir y enviar la respuesta de vuelta al cliente, permitiendo enviar datos en formato JSON o texto.
3.

Tipo de Middleware	Alcance	Características
Ruta específica	Una ruta determinada	Solo se ejecuta cuando la petición coincide con la URL y el método HTTP definidos. Se usa para validaciones puntuales o autenticación.
Manejo de errores	Toda la aplicación	Se activa únicamente cuando ocurre un fallo previo. Se distingue por recibir cuatro parámetros: `(er,req,res,next)`

4. Es un middleware de Express que toma el texto en formato JSON que envía el cliente en el cuerpo de la petición y lo convierte en un objeto JavaScript accesible mediante req.body.
   Si lo omitimos y el cliente envía información en formato JSON y este middleware no está activo, req.body llegará como undefined. Esto manda errores en el servidor al intentar acceder a los datos de la solicitud.
5. El orden es fundamental porque Express procesa las instrucciones en secuencia, de arriba hacia abajo, si se define una ruta antes que un middleware, esa ruta no ejecutará las acciones del middleware, también si configuras express.json() después de las rutas que reciben datos, la

aplicación intentará leer el cuerpo de la petición antes de convertirlo, además el middleware de manejo de errores debe ubicarse al final de todo el código para poder capturar las fallas emitidas por las rutas anteriores.

6. Si un middleware no finaliza el proceso con una respuesta , ni invoca la función next(), la petición queda bloqueada en el servidor. El cliente permanecerá esperando indefinidamente hasta que la conexión expire por timeout, afectando la experiencia del usuario y consumiendo recursos del servidor.
7.
    - **Middleware**: Su objetivo es realizar tareas intermedias, como verificar datos o comprobar permisos. Una vez completada su tarea, envía al siguiente paso.
    - **función de la ruta:** Es el punto final del flujo. Se encarga de procesar la lógica principal y enviar la respuesta al cliente, terminando el ciclo.

8. Omitir estos controles genera vulnerabilidades en la aplicación:

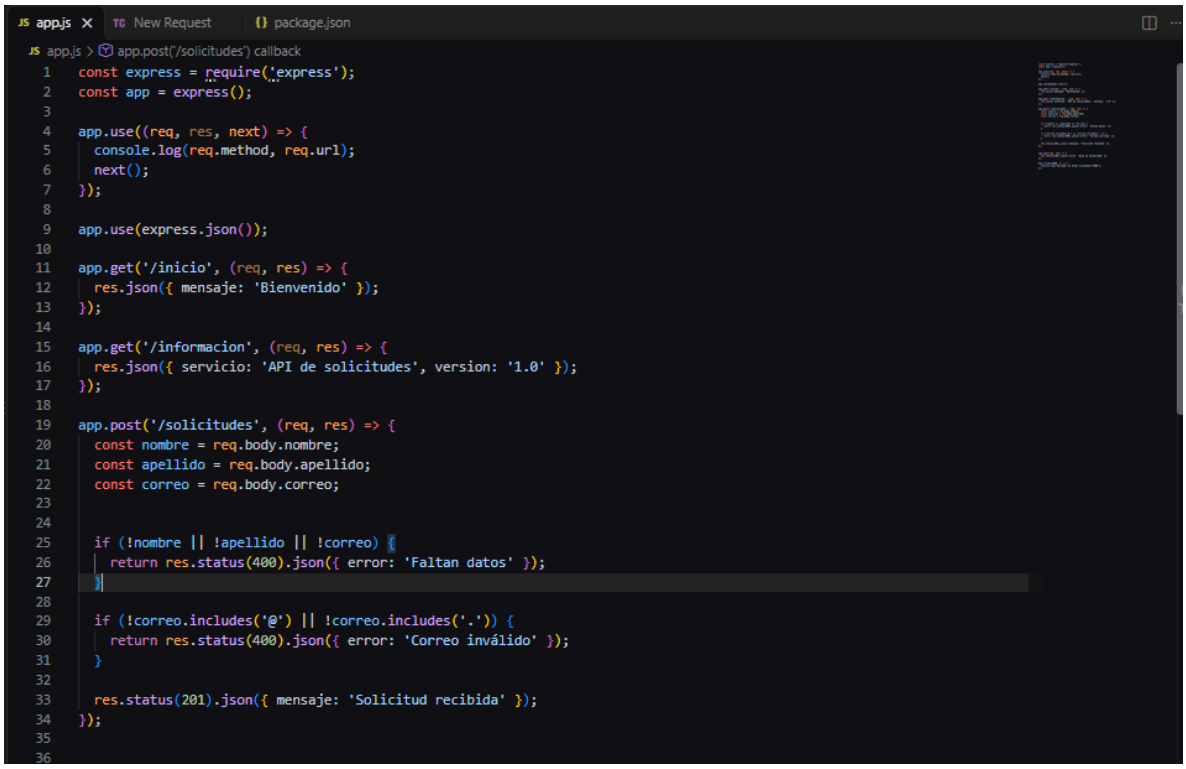
    - Usuarios no autorizados podrían acceder, modificar o eliminar información restringida.

    - La falta de validación expone la base de datos a ataques como SQL o NoSQL Injection.

    - El ingreso de datos incompletos o con formato incorrecto puede generar errores no controlados que detengan el funcionamiento de la API.
9. **ejemplo:**
    - **Filtro de seguridad inicial o Middleware Global:** El personal en la entrada revisa que todos los asistentes pasen por el detector de metales, esto es para todo el publico.
    - **Control de acceso a zona VIP o Middleware de Ruta Específica:** Al intentar ingresar a la zona VIP, un guardia verifica que tengas la credencial correspondiente. Si la tienes, te permite avanzar, de lo contrario, detiene su paso e indica que no puedes ingresar, enviando una respuesta de rechazo e interrumpiendo el flujo.
    - **El espectáculo o Función de la ruta:** Es el destino final del asistente, donde disfruta del evento y concluye la experiencia o se procesa la solicitud y se entrega la respuesta final.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

5. Construcción del middleware propio:



```
const express = require('express');
const app = express();

app.use((req, res, next) => {
  console.log(req.method, req.url);
  next();
});

app.use(express.json());

app.get('/inicio', (req, res) => {
  res.json({ mensaje: 'Bienvenido' });
});

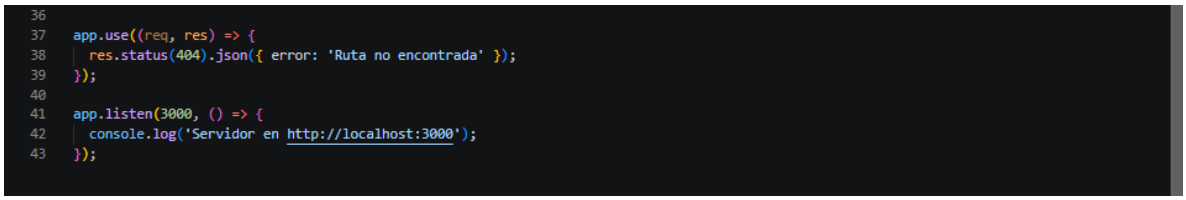
app.get('/informacion', (req, res) => {
  res.json({ servicio: 'API de solicitudes', version: '1.0' });
});

app.post('/solicitudes', (req, res) => {
  const nombre = req.body.nombre;
  const apellido = req.body.apellido;
  const correo = req.body.correo;


  if (!nombre || !apellido || !correo) {
    return res.status(400).json({ error: 'Faltan datos' });
  }

  if (!correo.includes('@') || !correo.includes('.')) {
    return res.status(400).json({ error: 'Correo inválido' });
  }

  res.status(201).json({ mensaje: 'Solicitud recibida' });
});
```



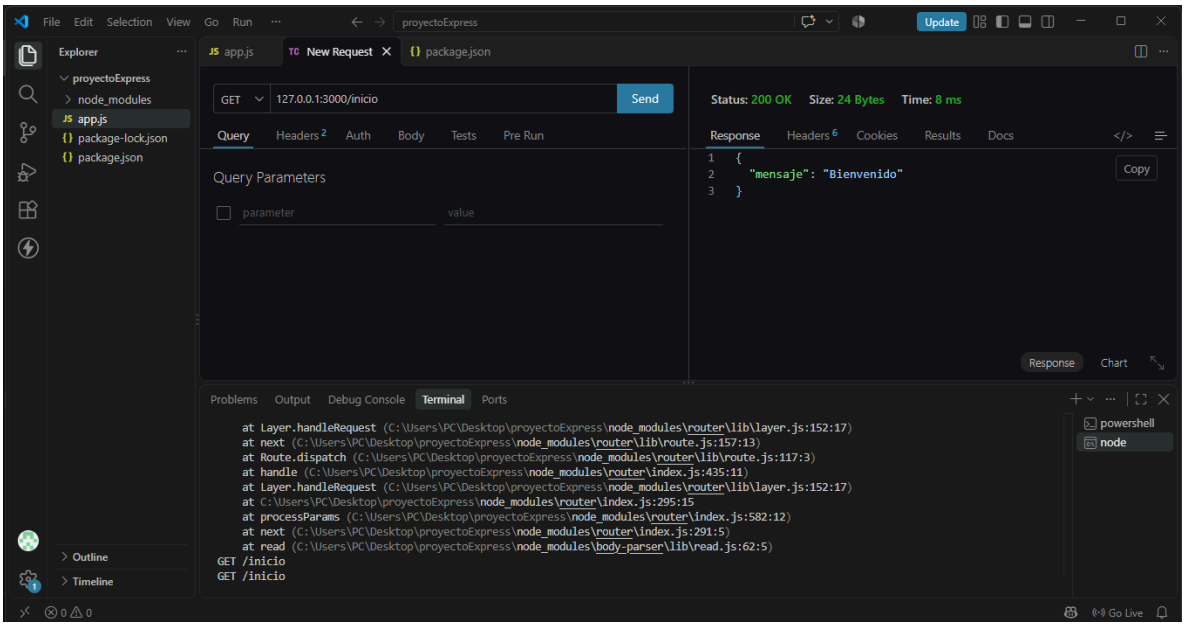
```
app.use((req, res) => {
  res.status(404).json({ error: 'Ruta no encontrada' });
});

app.listen(3000, () => {
  console.log('Servidor en http://localhost:3000');
});
```

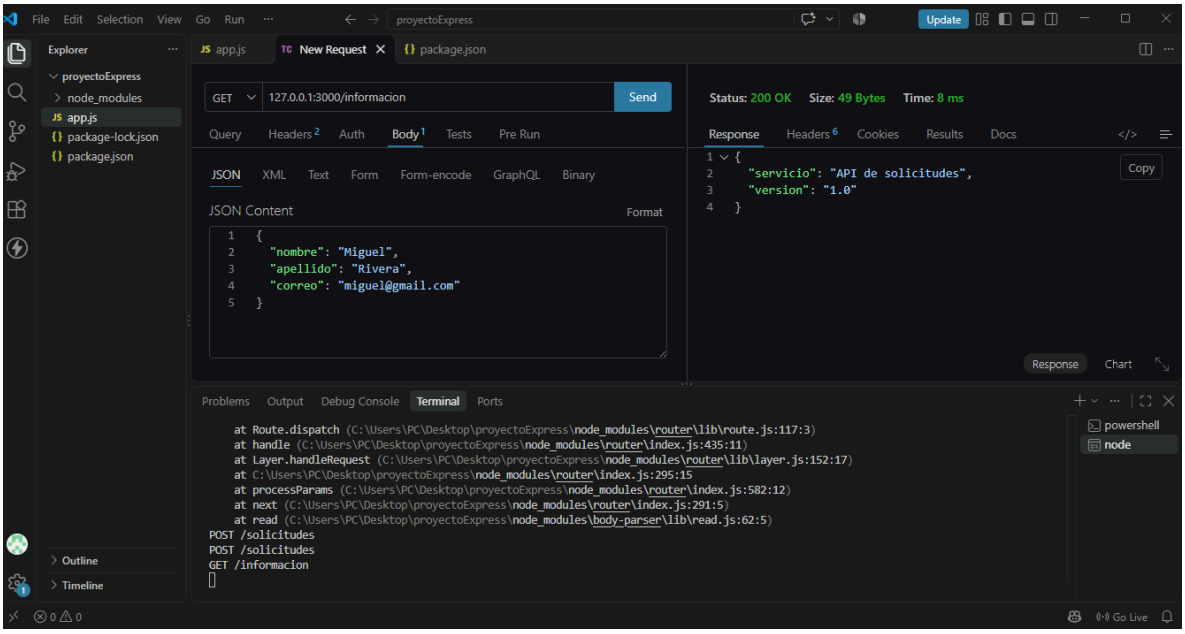
- **Registro generado por las rutas get:**

  inicio:

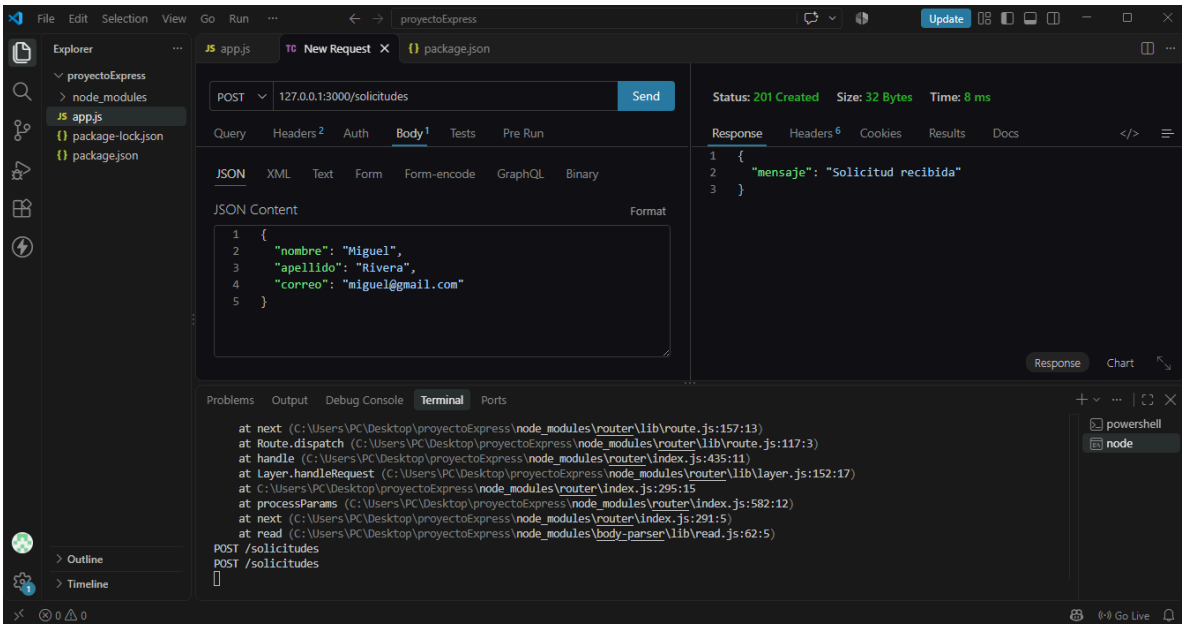
PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS



información:



- **Registro generado al enviar una petición POST:**

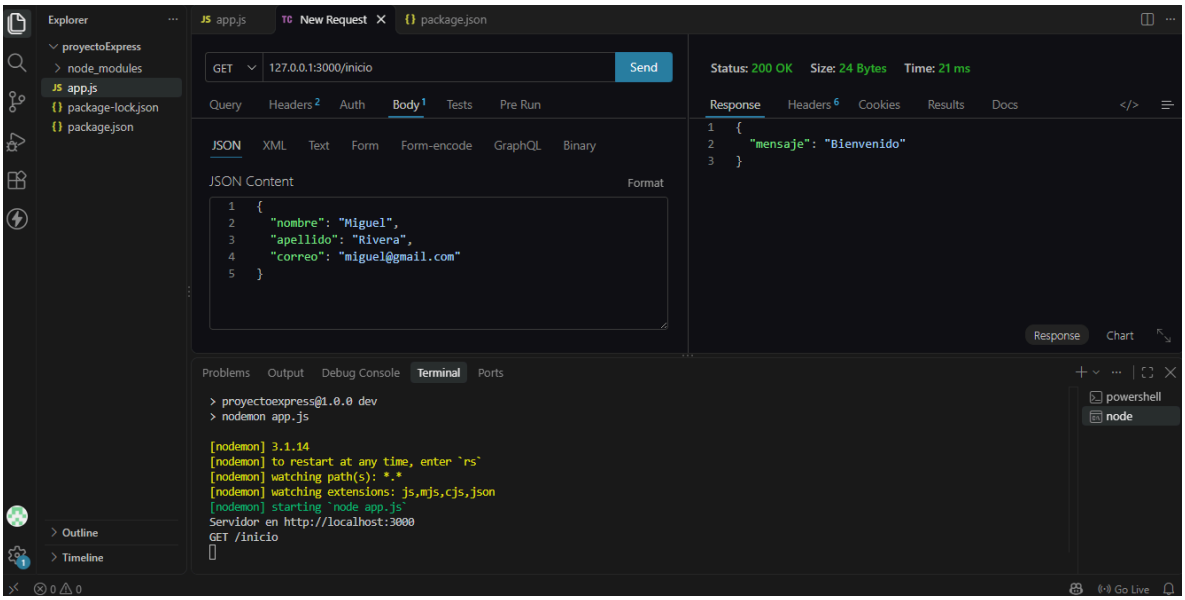


- Express revisa en orden. Si una ruta ya respondió, la petición termina ahí y nunca llega al middleware que está más abajo. lo que haría que no se registrará nada de /inicio, /informacion ni /solicitudes. Solo registraría rutas que no existen, y en ese caso el 404 también lo cortaría, así que no registraría nada.
- Con next(). Cuando el middleware termina de registrar, llama a next() y Express pasa al siguiente middleware (express.json()) y luego a la ruta que coincida. Si no se llama a next() ni se responde, la petición queda colgada.
- Lo puse al inicio para que registre todas las peticiones, antes de que alguna ruta pueda responder y cortar el flujo.

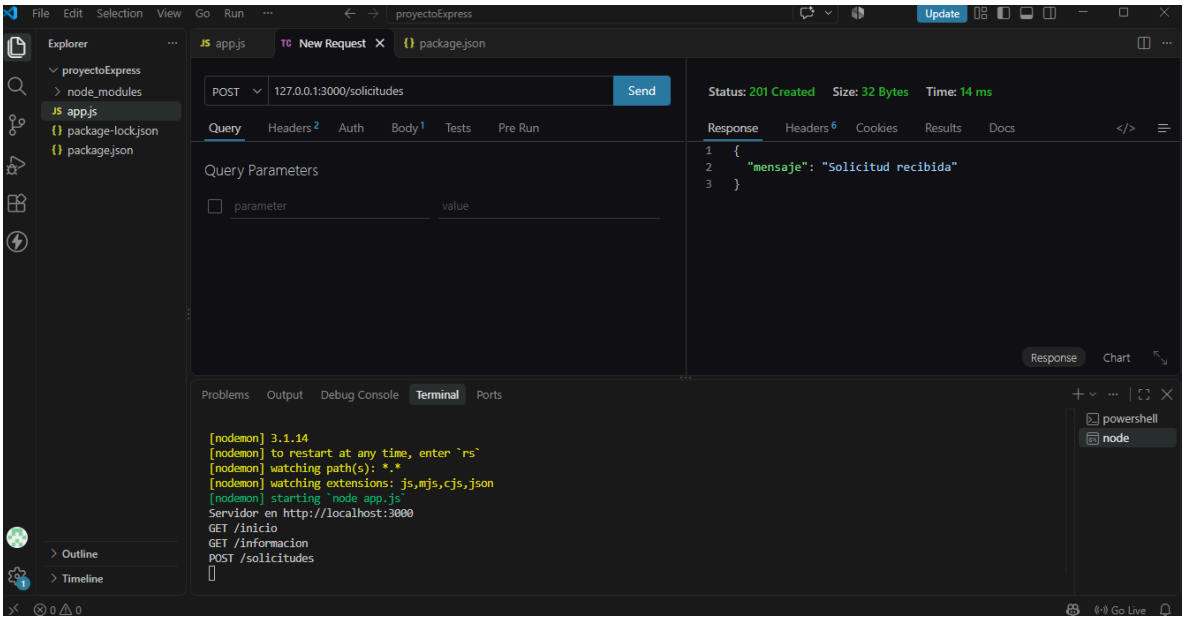
6. código de estado HTTP y manejo de errores:

- **200 OK:** la petición se procesó bien y solo se devuelve información. Lo usan GET /inicio y GET /informacion.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

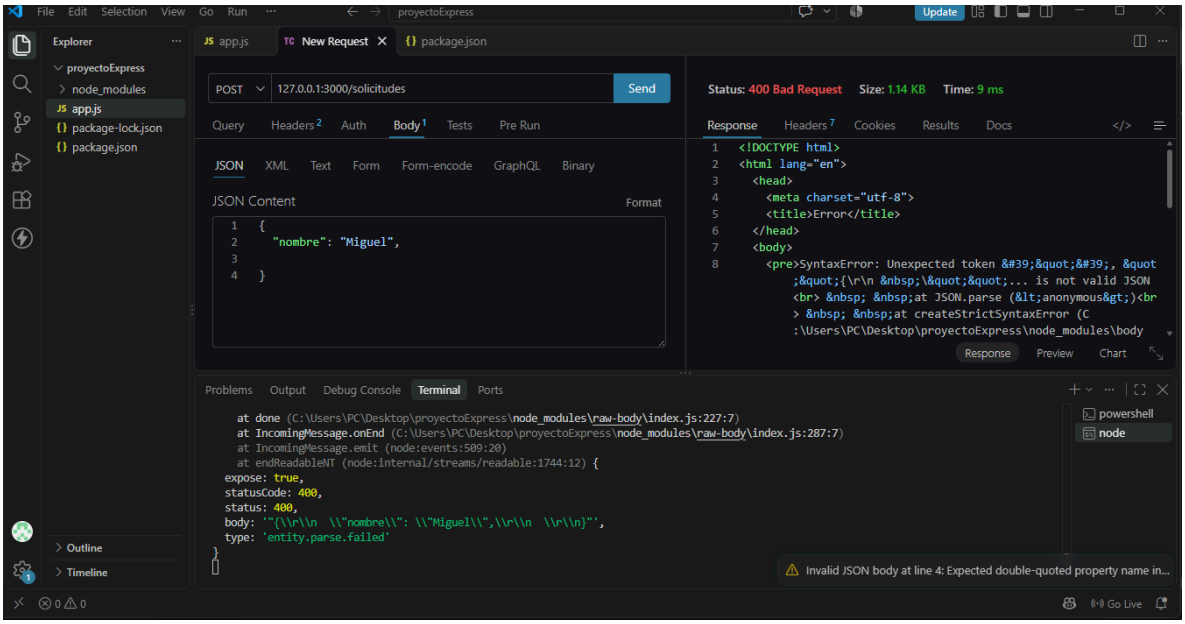


- **201 Created:** en POST /solicitudes, cuando los datos son válidos, se recibe una solicitud nueva. 201 indica que se creó o registró algo, no solo que se consultó.

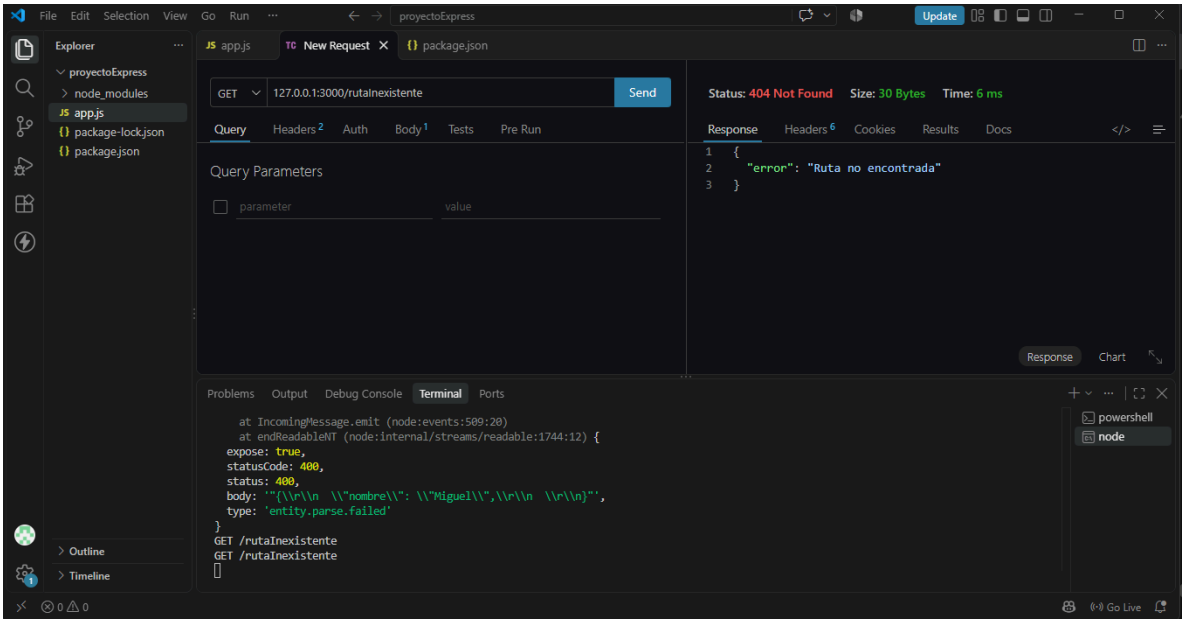


- **400 Bad request:** El error está en lo que envió el cliente: faltan campos o el correo no tiene formato válido. Responder 200 sería engañoso, porque el cliente creería que todo salió bien.

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

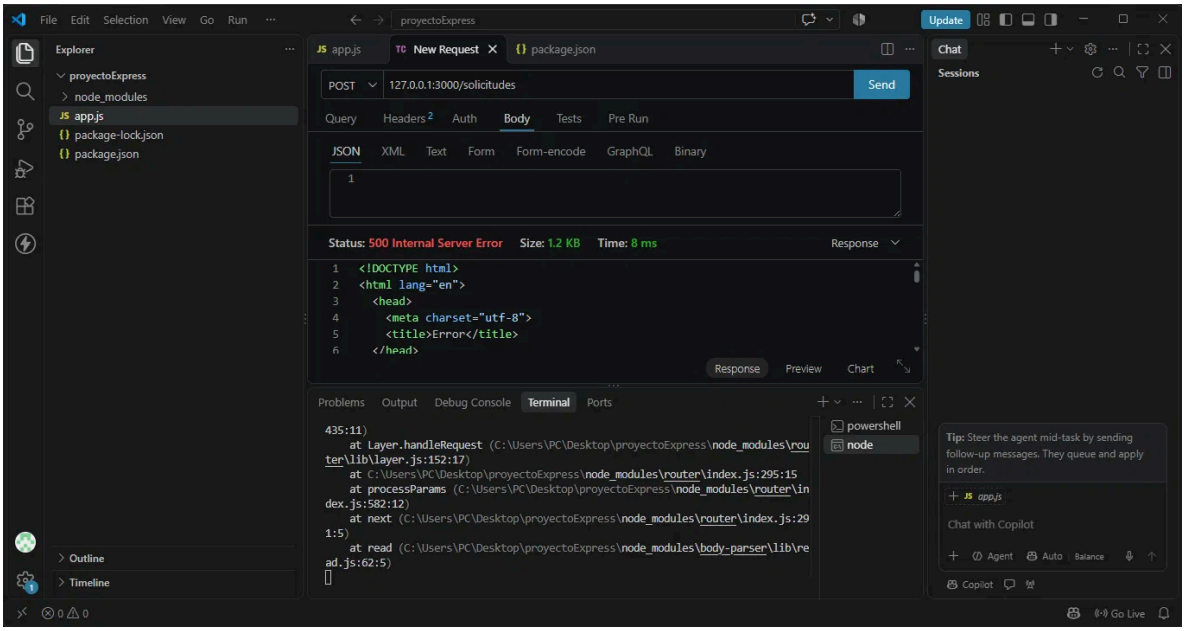


- **404 Not Found:** el cliente pidió una ruta que no existe en la API. Se responde al final del archivo, cuando ninguna ruta anterior coincidió.



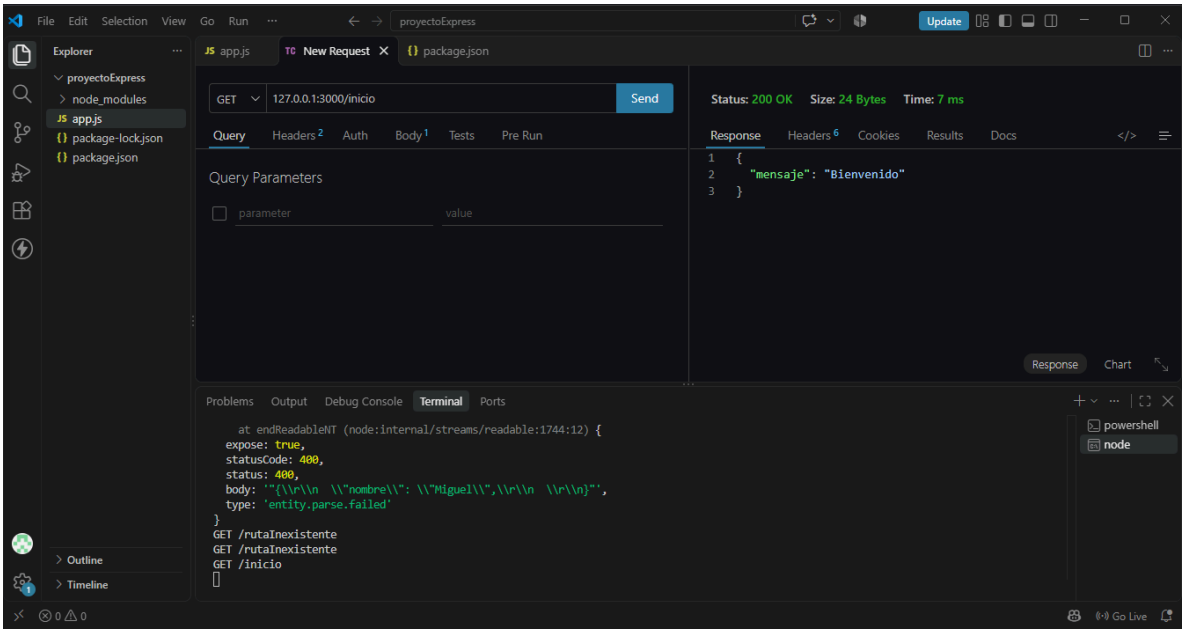
- **500 Internal Server Error:** corresponde cuando el fallo es del servidor, no del cliente: un error inesperado en el código, como

intentar leer una propiedad de algo que no existe. Se diferencia del 400 en que el cliente no hizo nada mal.



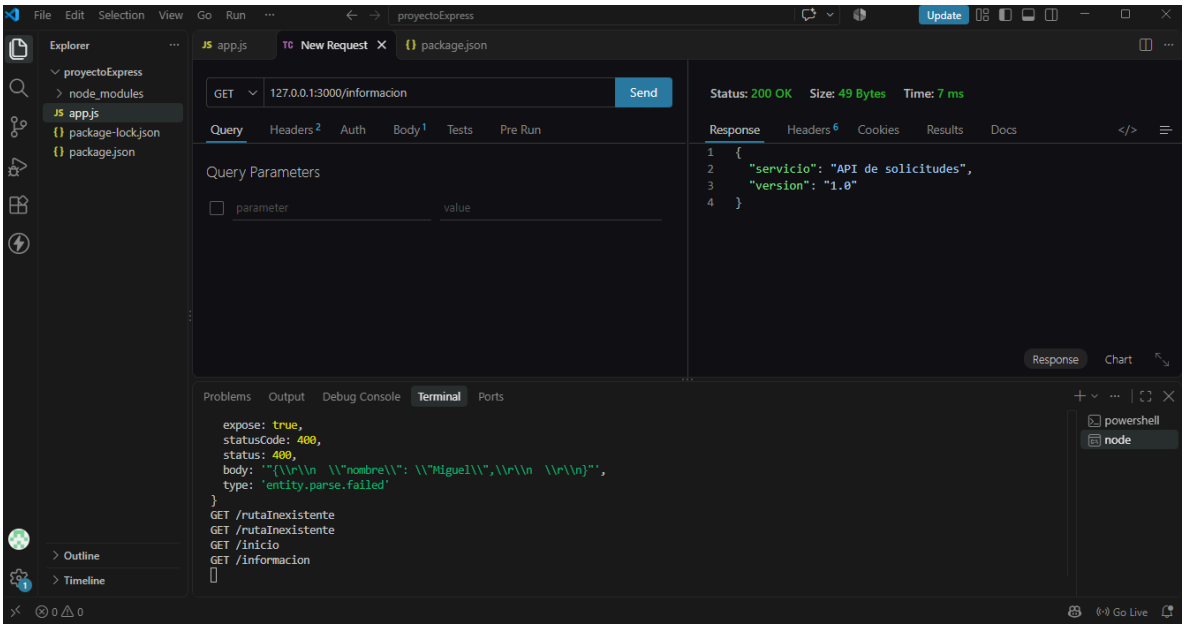
7.

- **GET /inicio:**

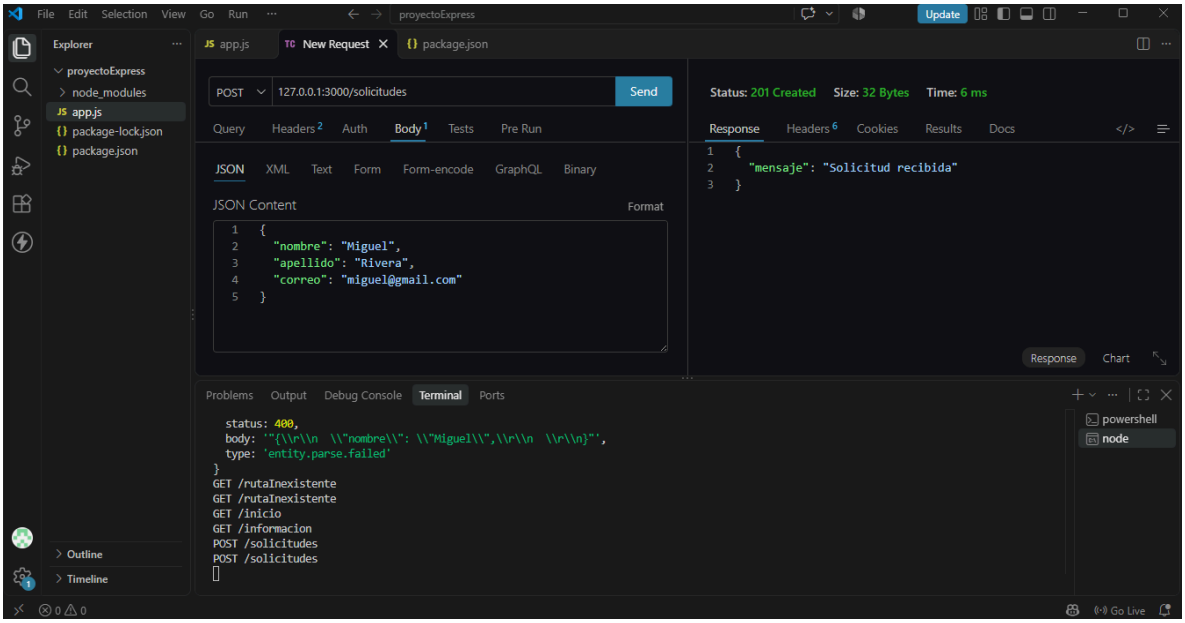


- **GET /informacion:**

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

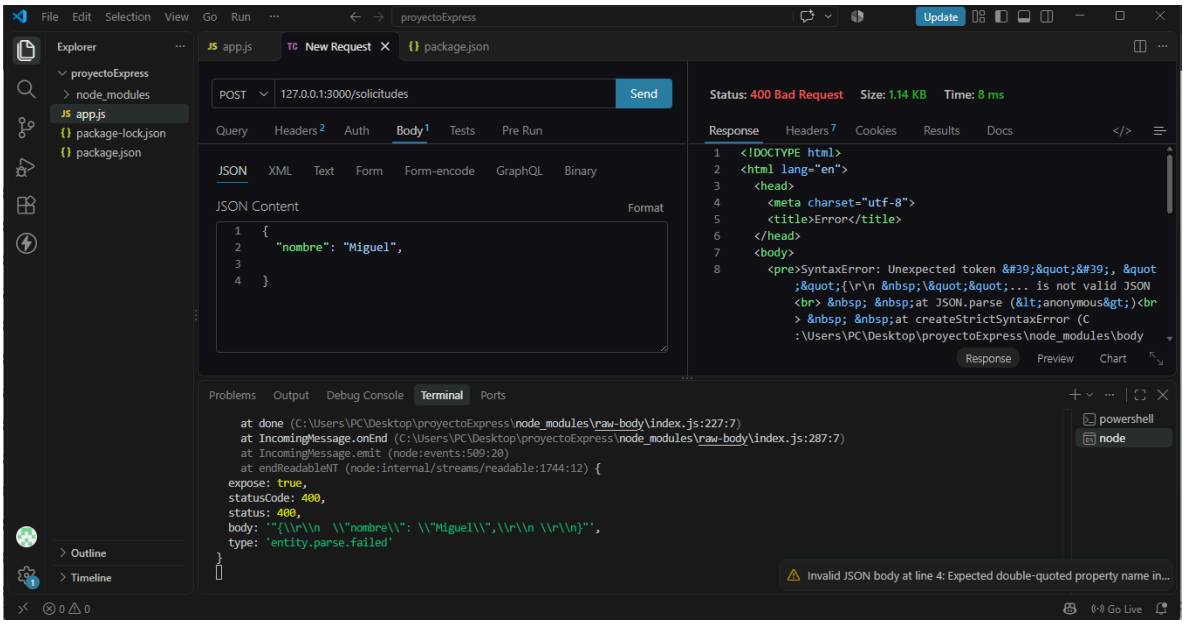


- **POST /solicitudes con nombre, apellido y correo válidos:**

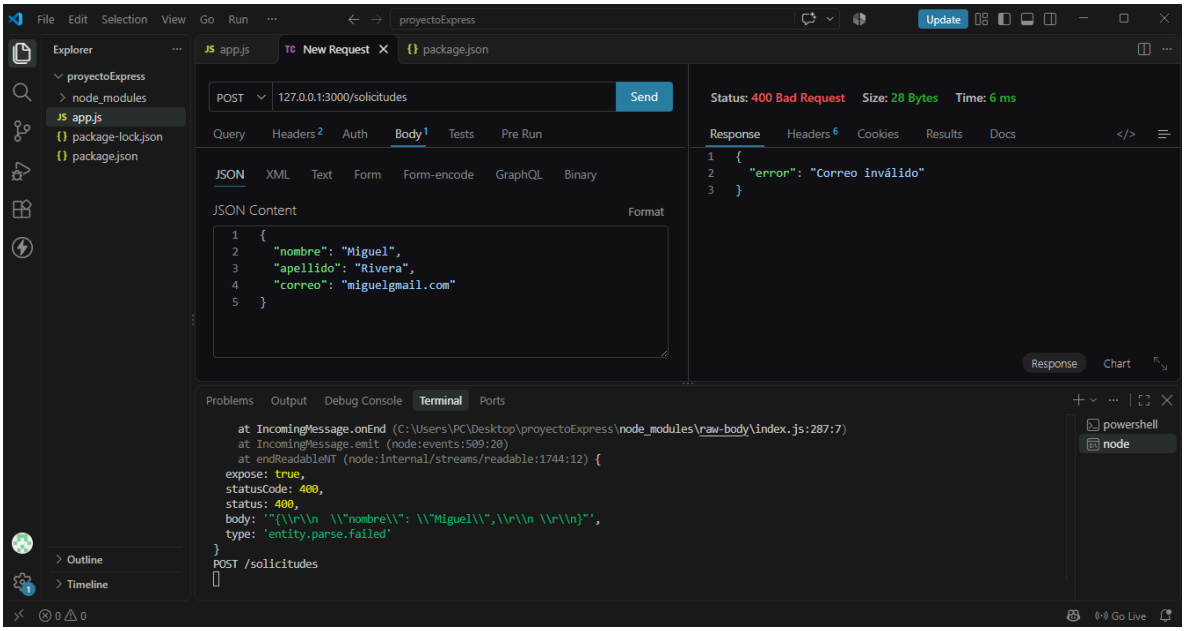


- **POST /solicitudes sin uno o más campos obligatorios:**

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS

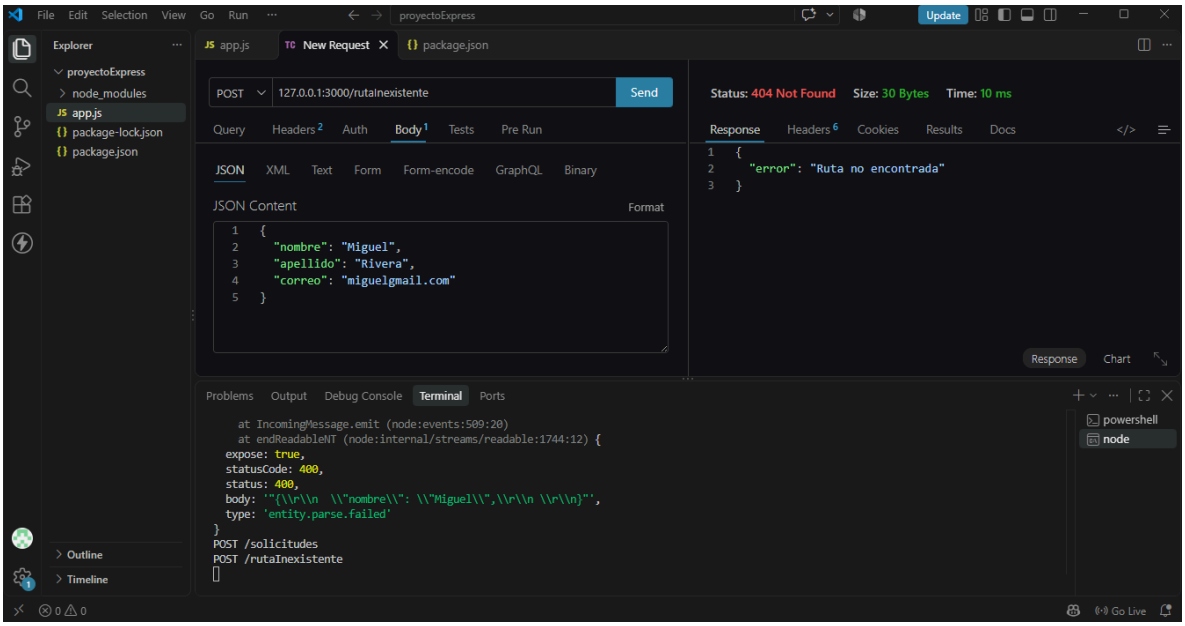


- **POST /solicitudes con correo de formato inválido:**

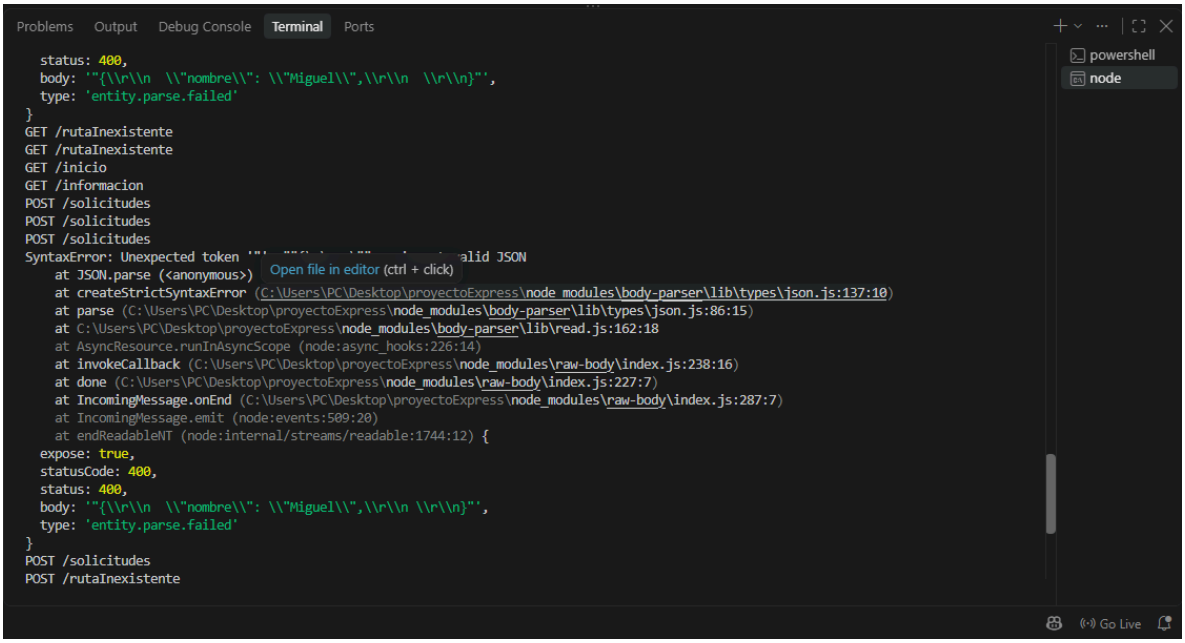


- **Consultar una ruta que no exista y verificar la respuesta 404:**

PROCESO DE GESTIÓN DE LA FORMACIÓN PROFESIONAL INTEGRAL
FORMATO ENTREGA DE EVIDENCIAS



- **Registro de peticiones con Middleware:**



- **Probar qué ocurre si se envía un cuerpo JSON y explicar el papel de express.json():** express.json() es un middleware que lee el cuerpo de la petición

cuando el cliente lo envía en formato JSON. Ese cuerpo llega como texto plano, y el middleware lo convierte en un objeto de JavaScript y lo guarda en req.body. Gracias a eso la ruta puede usar req.body.nombre, req.body.correo, etc.